Pay — Architecture & Flows

> A plain-English map of **how this product works, why each piece is built the way it is, what legal/compliance question it touches, and how finished it is.** Read top-to-bottom for the full picture, or jump to the integration you care about.

The product is a **consumer money app** — you hold a dollar balance, send money to friends, and pay merchants — built on **stablecoins (USDC) on Solana devnet**, with the crypto deliberately hidden. The north star (`stablecoin-onofframp-architecture.pdf`) is *"a money product with protocol advantages: expose certainty, hide complexity, failure-first then growth."*

How to read these docs

Every integration/flow doc follows the **same six-part shape**, so you always know where to look:

Section	Answers
What it is	one-paragraph definition
Flow	a diagram of how it actually moves
Why this way	the design decision + the tradeoff
Legalities	the regulatory question it touches, and how we handle or *defer* it
Status	how done it is — ✅ complete · 🟡 gray (partial/simulated) · ⬜ planned
Key files	where it lives in the repo

Status legend

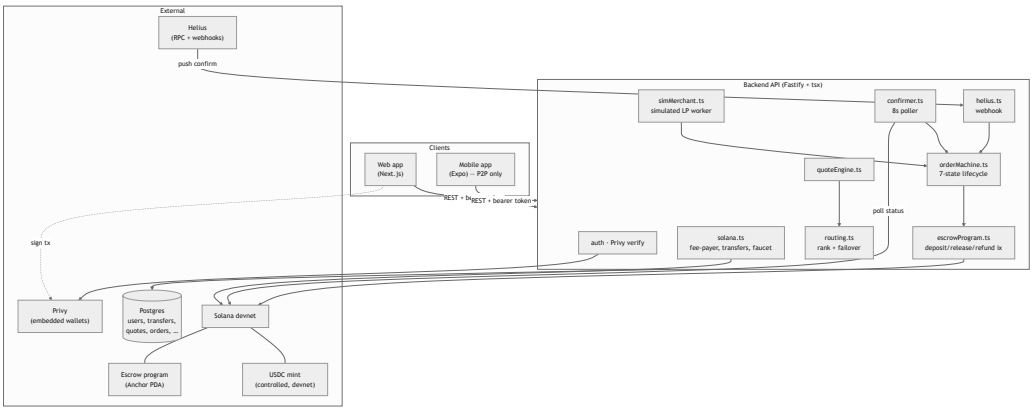
- ✅ **Complete** — built, wired, and verified (at least headlessly).
- 🟡 **Gray** — exists but **simulated, partial, or cosmetic** (not production-real).
- ⬜ **Planned** — designed, not yet built.

The docs

#	Doc	Covers	Status
01	Architecture	the whole system, monorepo layout, request + on-chain lifecycle	✅
02	Auth & identity	Privy email login, embedded wallet, `@handle`, enforced KYC tiers	✅ / 🟡 vendor
03	Custody & gas	self-custody + fee-payer gas sponsorship (the key legal lever)	✅
04	Money in	funding the balance (faucet today → card/bank later)	🟡
05	P2P send & request	send money to a friend, request money	✅
06	Payout order lifecycle	quote engine + the 7-state order machine	✅
07	On-chain escrow	the Anchor program that holds funds (M2)	✅
08	Routing, scoring & the LP marketplace	ranking, silent failover (M3) + live signals/float/KYB/bonds/disputes (M4)	✅
09	Confirmation pipeline	how the chain tells the app a payment landed	✅
10	Data model	the Postgres schema, table by table	✅
11	Compliance & legalities	the honest risk map: custody, KYC, MTL, FX, sanctions	🟡
12	Status & roadmap	the M0→M7 heat-map of done vs. gray vs. planned	—
13	Future milestones	M7: real rails (the last milestone)	⬜
14	Notifications & infra	in-app inbox, advisory lock, rate-limit, metrics (M6)	✅ / 🟡 scale

---

The system at a glance



One-line mental model: the **client** asks the **backend** to *build* a transaction, the **user** signs it (via Privy), the **fee-payer** sponsors the gas, the **chain** executes it, and a **confirmer + webhook** tell the backend it landed so the **order machine** can advance. Money that's "in flight" for a merchant payout sits in an **on-chain escrow** until it's released or refunded.

See 01-architecture for the detailed walk-through.

The PDF & regenerating it

`payments-architecture.pdf` is the whole set as one diagram-rich document (~37 pages). It's built from the Markdown with **no installed packages** — `marked` + `mermaid` load from a CDN at view time:

```
# 1. assemble the self-contained, print-ready HTML (Node only, no npm install)
node docs/build-architecture-html.mjs

# 2a. render to PDF with an already-installed Chrome (no package install):
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
  --headless --disable-gpu --virtual-time-budget=30000 --no-pdf-header-footer \
  --print-to-pdf="$(pwd)/docs/payments-architecture.pdf" \
  "file://$(pwd)/docs/architecture.html"

# 2b. …or just open docs/architecture.html in any browser → Print → Save as PDF.
```

Edit the `.md` files (the source of truth), then re-run step 1 (+2) to refresh the PDF.

01 · Architecture

What it is

A **yarn-workspaces monorepo** with four apps and two shared packages. The backend is the brain (it builds transactions, runs the order lifecycle, talks to the chain and the database); the clients are thin (they render, collect a signature, and poll). Everything is **always-real against Solana devnet** — there is no mock toggle; "simulation" only ever means a *simulated merchant/LP*, never a fake chain.

```
monorepo/
├── apps/
│   ├── backend/     Fastify API (tsx, ESM) — the brain
│   ├── web/         Next.js consumer app (full feature set incl. cash-out)
│   └── mobile/      Expo app (P2P only today; no order flow yet)
├── packages/
│   └── shared/      @pay/shared — TypeScript types + money helpers (RATE, inr(), usd())
└── anchor/          Solana program (Rust + Anchor) — the on-chain escrow
```

Layer	Tech	Notes
Backend	**Fastify 5 + tsx** (no build step), ESM	one process; a background poller runs inside it
Web	**Next.js App Router + Zustand + Privy web SDK**	mobile-first browser UI
Mobile	**Expo + Privy Expo SDK**	consumer P2P; cash-out flow not ported
Shared	raw TS (`main: ./src/index.ts`)	backend imports **types only** (see below)
Chain	**Solana devnet, SPL "USDC"** (6 dp), Anchor 0.32	controlled devnet mint in dev
Data	**Postgres** (Railway/Supabase)	access control in the API layer, no RLS
Infra	Privy (wallets), Helius (RPC + webhooks)	both have free tiers

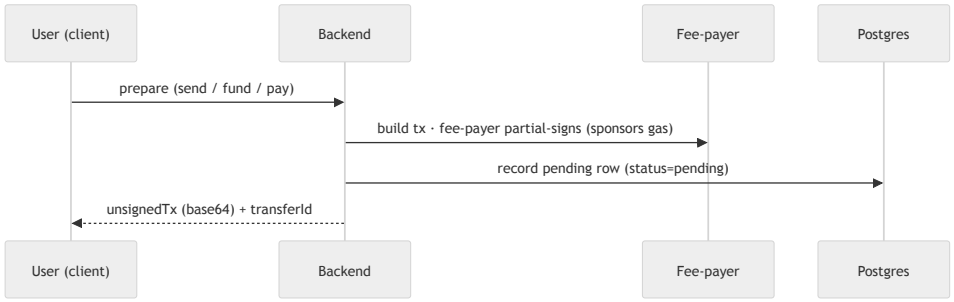
> **Quirk worth knowing:** the backend imports from `@pay/shared` as **types only** (`import type`). Runtime *value* imports from the raw-TS workspace package fail under `tsx` , so shared constants (e.g. the quote fee constants in `quoteEngine.ts`) are **mirrored locally** in the backend rather than imported. If you add a shared runtime value the backend needs, copy it, don't import it.

Flow — the universal "build → sign → submit → confirm" lifecycle

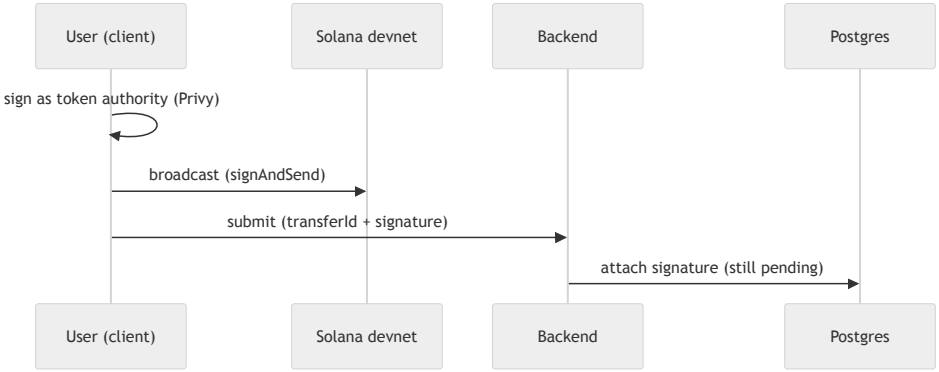
Almost every money action (send, fund-escrow, faucet) follows the **same four beats**. The backend never holds the user's key; it *prepares* a transaction, the *user* signs it, and the fee-payer *sponsors* the gas.

The four beats are shown as four small diagrams so each stays readable.

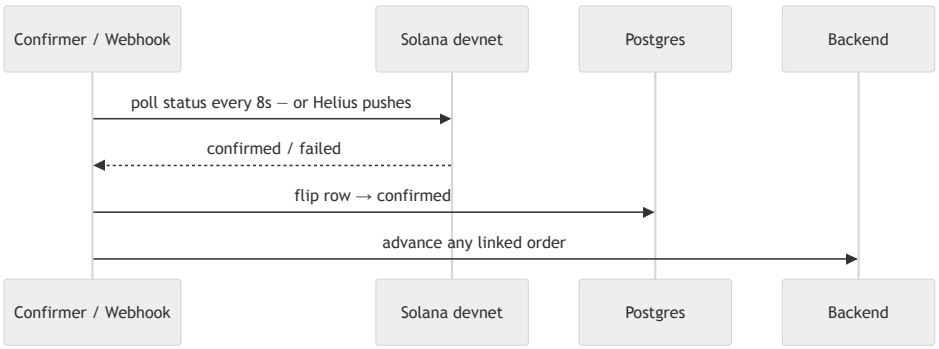
1 · Build (prepare)



2 · Sign & submit

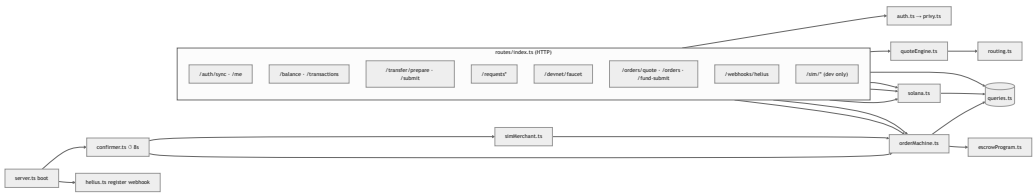


3 · Confirm (async)



The "prepare/submit seam" (`solana.ts` comment) is the whole trick: because the client signs an already-fee-payer-sponsored transaction, **users never need SOL** and the platform **never custodies the user's key**. The same seam carries the on-chain escrow `deposit` unchanged — see 03-custody-and-gas.

Component map



Who calls whom, in words:

- `server.ts` boots Fastify, registers routes, starts the **confirmer** poller (8s), and registers the **Helius** webhook (no-op locally).

- **`routes/index.ts`** is the only HTTP surface. An `authPreHandler` populates `req.userId` from a Privy bearer token (or `x-dev-user-id` in dev).
- **`solana.ts`** owns the chain primitives: `connection()`, `feePayer()`, `buildUsdcTransfer`, `mintUsdcTo` (faucet), `getUsdcBalance`, `getTransferStatus`.
- **`escrowProgram.ts`** owns the Anchor program: `buildDepositTx`, `releaseEscrow`, `refundEscrow`.
- **`quoteEngine.ts` → `routing.ts`** price a quote and pick a merchant.
- **`orderMachine.ts`** runs the 7-state order lifecycle and is the *only* writer of order state.
- **`confirmer.ts`** (every 8s) confirms pending transfers **and** ticks **`simMerchant.ts`** (the simulated LP + escalation timers).
- **`queries.ts`** is the single SQL module (no ORM).

Why this shape

- **Thin clients, fat backend.** Pricing, state, and chain logic live server-side so web + mobile stay simple and consistent, and so the order machine has one authoritative writer.
- **No build step (tsx).** Fast iteration for a solo builder; the cost is the types-only `@pay/shared` quirk above.
- **One poller inside the API process.** Rate-limiting, metrics, and notifications are in-process (M6); a separate worker/Redis (multi-instance) is the remaining M7 scale step. The 8s confirmer doubles as the order-escalation clock. Simple, good enough for devnet, explicitly temporary.
- **Always-real, devnet.** Avoids mock/real drift: the code you test is the code that ships; only the *counterparty* (merchant/LP) is simulated.

Legalities (system-level)

The architecture is deliberately **non-custodial** (user signs; platform sponsors gas) — this is the single biggest regulatory lever, because **not holding customer funds or keys** keeps us out of the heaviest money-transmitter / custody obligations during the build phase. The one place funds *do* sit under program control is the **per-order escrow** (07), and even there the user authorizes the deposit and the program enforces release/refund rules on-chain. Full detail in 11-compliance-and-legalities.

Status

✅ The architecture itself is complete and stable: monorepo, backend services, web app, shared types, the build→sign→submit→confirm seam, and the on-chain escrow are all wired and typecheck-clean across 4 workspaces. 🟡 Gray spots are *what flows through it* — a simulated LP network, a faucet instead of a real on-ramp, and a simulated KYC vendor (live FX + enforced KYC tiers already shipped) — each called out in its own doc.

Key files

- `apps/backend/src/server.ts` · `routes/index.ts` · `config.ts`
- `apps/backend/src/services/{solana,escrowProgram,quoteEngine,routing,orderMachine,simMerchant,confirmer,helius}.ts`
- `apps/backend/src/db/{schema.sql,queries.ts}`
- `packages/shared/src/{contracts.ts,money.ts}`
- `anchor/programs/escrow/src/lib.rs`

02 · Auth & Identity

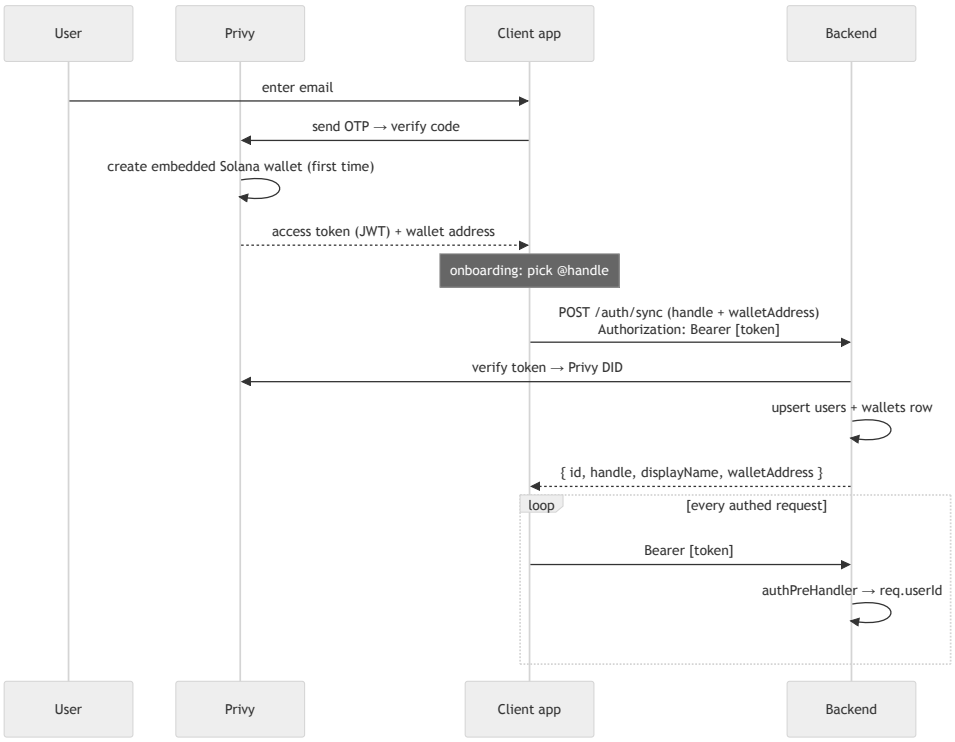
What it is

How a person becomes a user. **Privy** provides email-OTP login and silently creates an **embedded Solana wallet** for them (no seed phrase, no "connect wallet"). The user picks an `@handle` (like Venmo/Cash App) so friends can pay them by name, not by address. Identity verification (**KYC**) is modelled as tiers that **enforce** money-movement limits (M5; see Status below), backed by a simulated verification vendor for now.

Three identities are tracked per person:

- **Privy DID** — the login identity (`did:privy:…`), the source of truth for "who is this request."
- **DB user id** — our internal `users.id` , created on first `/auth/sync` .
- `@handle` — the human-facing name; unique, used for resolve/pay.

Flow — login → sync → authenticated requests



On the backend (`auth.ts`): an `authPreHandler` runs before every route. In **Privy mode** it verifies the `Authorization: Bearer` JWT (`verifyAccessToken`), maps the **Privy DID → DB user**, and sets `req.userId` . Before the user exists, `/auth/sync` uses `req.privyDid` to create them. In **dev mode** (Privy unconfigured), it falls back to an `x-dev-user-id` header so the API is testable without Privy. `requireUserId(req)` throws 401 if unmapped.

Why this way

- **Email + embedded wallet, not "connect a wallet."** The target user has no wallet and shouldn't need one. Privy's embedded wallet means **no seed phrase ever touches the user** — onboarding feels like any fintech app ("No passwords, just your email").
- `@handle` **over address.** Nobody pays a base58 string. Handles are the entire P2P UX; addresses are resolved server-side and never shown.
- **DID as the spine.** Tying our user to the Privy DID (not email, which can change) keeps identity stable and lets Privy own the sensitive auth surface.

- **Tiered KYC as a gate, not a wall.** The design lets low-value P2P work at tier 0–1 and only asks for documents when the user does something that legally needs it (paying merchants) — "progressive KYC," JIT.

Legalities

- **KYC/AML (Customer Identification).** The **tiered** model (caps scale with verification) is now **enforced** (M5): `users.kyc_tier` (0–3) gates money movement — tier 1 caps P2P at $100/tx, tier 2 unlocks merchant pay — checked server-side against rolling-24h ledger volume, with a simulated verification upgrade (`/kyc/verify`) and sanctions screening. **Still simulated** for the *vendor* leg: no real document collection / KYC-AML provider — that seam (Persona/Onfido/Sumsub) lands at **M7**.
- **Custody of credentials.** By delegating auth + key management to Privy, we avoid storing passwords or private keys ourselves — smaller breach surface and smaller compliance burden. See 03-custody-and-gas.
- **Data/PII.** `users` stores email/phone/handle. Minimal today; a real launch needs a privacy policy, data-retention rules, and regional handling (e.g. India DPDP, GDPR).

Status

- ✅ Privy email-OTP login + embedded wallet creation, `@handle` selection, `/auth/sync` , bearer-token verification, dev `x-dev-user-id` fallback.
- ✅ **KYC tiers enforced (M5)** — `kyc_tier` per-tier caps enforced at the money-movement endpoints (structured `limit_exceeded`), a simulated verification upgrade, `/me` returns the tier, sanctions screening + a fraud score, and the Verify UI reflects the real tier.
- 🟡 **Vendor leg still simulated** — no real document capture / KYC-AML provider (M7).

Key files

- Backend: `apps/backend/src/auth.ts` , `services/privy.ts` , route `/auth/sync` + `/me` in `routes/index.ts` , `db/queries.ts` (`upsertUserByPrivyDid` , `findUserByPrivyDid`).
- Web: `src/services/privy/PrivyBridge.tsx` , `app/onboarding/{faceid,handle}/page.tsx` , `components/sheets/identity.tsx` (the KYC/recovery sheets).
- Schema: `users` , `wallets` (see 10-data-model).

03 · Custody & Gas Sponsorship

> This is the **single most important page for understanding the product's legal posture.** The custody model is what keeps us out of the heaviest regulation while we build.

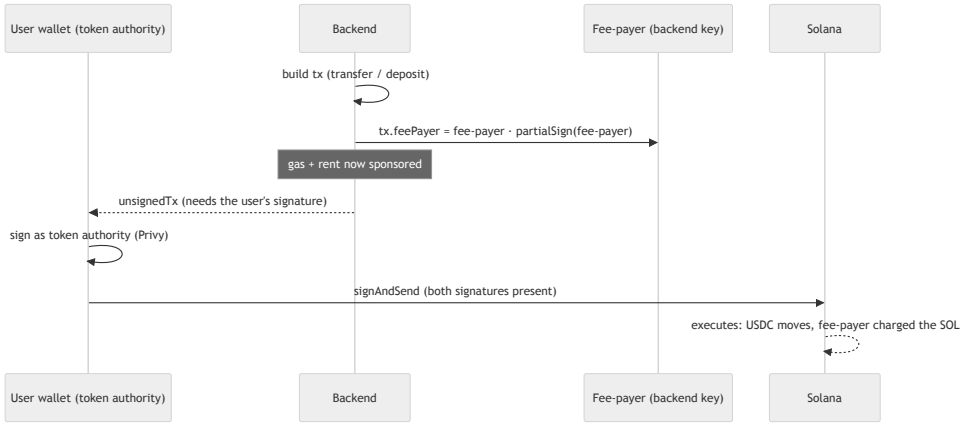
What it is

Two linked decisions:

1. **Self-custody.** The user's funds live in *their own* embedded wallet (a Solana account whose key Privy manages for them). The platform **never holds the user's private key** and **never moves their money without their signature.**
2. **Gas sponsorship (fee-payer model).** Solana transactions cost a tiny amount of SOL in fees. Users don't have SOL and shouldn't have to. So a backend **fee-payer** account pays the network fee (and any account-creation rent) on every transaction. The user only ever signs as the **token authority** (the owner of the USDC being moved).

The result: the user experiences "tap to pay," while under the hood **they** authorize the movement and **we** pay the gas.

Flow — who signs what



Two signatures, two roles:

- **Fee-payer** signs to say *"I'll pay the network fee."* (added server-side, `partialSign`)
- **User** signs to say *"I authorize moving my USDC."* (added client-side, via Privy)

Neither can act alone: the fee-payer can't move the user's tokens (it isn't the token authority), and the user can't broadcast a fee-less tx (no SOL). This is enforced by Solana itself.

In code (`solana.ts`):

```
tx.feePayer = payer.publicKey;
tx.recentBlockhash = (await connection().getLatestBlockhash()).blockhash;
tx.partialSign(payer);                 // fee-payer sponsors
// → client adds the user (token-authority) signature and broadcasts
const unsignedTx = tx.serialize({ requireAllSignatures: false }).toString('base64');
```

The fee-payer is also the **mint authority** for the controlled devnet USDC (so it can run the faucet, 04) and the **escrow program authority** (so it can release/refund, 07). Its secret is `FEE_PAYER_SECRET_KEY` , loaded once and cached.

Why this way

- **No SOL for users = no dead-end.** The #1 reason normal users churn out of crypto apps is "now buy some gas token." Sponsorship removes it entirely.
- **Self-custody = smaller legal + security surface.** We can't lose what we don't hold. No key vault, no "we got hacked and lost customer funds," and a much lighter regulatory classification (see Legalities).
- **One seam for everything.** Send, fund-escrow, and faucet all reuse the same prepare→sign→submit shape, so the on-chain escrow `deposit` (M2) dropped in with **near-zero client change** — the client signer is transaction-agnostic.

Legalities

- **Money transmission / custody.** Holding customer funds typically triggers **money-transmitter licensing** (US state MTLs / FinCEN MSB), an EU **VASP/EMI** registration, or equivalent. A **non-custodial** design — where the user holds and authorizes their own funds — generally carries a **materially lighter** classification. By never holding the user's key, we stay on the lighter side during the build phase. *(Not legal advice; the line is fact-specific and varies by jurisdiction — this must be confirmed with counsel before real funds move at M7.)*
- **The escrow caveat.** The one place value sits under *program* control is the per-order escrow. It's designed to be **trust-minimized** (the user authorizes the deposit; the program's rules govern release/refund; funds are either paid to the merchant or returned to the user), which keeps it closer to "conditional settlement" than "custody." Still, **who controls the release authority** (today: our fee-payer) is the key legal question for that flow — detailed in 07 and 11.
- **Gas sponsorship.** Paying users' network fees is a product cost, not a regulated activity — but at scale it's an abuse vector (someone could drain the fee-payer with spam), so M6 adds rate-limiting.

Status

- ✅ Self-custodial embedded wallets; fee-payer gas + rent sponsorship on every transaction; the prepare→sign→submit seam; fee-payer doubling as mint + escrow authority on devnet.
- 🟡 The fee-payer is a **single hot key** in env config — fine for devnet, but production needs an HSM/KMS or multisig and spend controls (M6/M7).

Key files

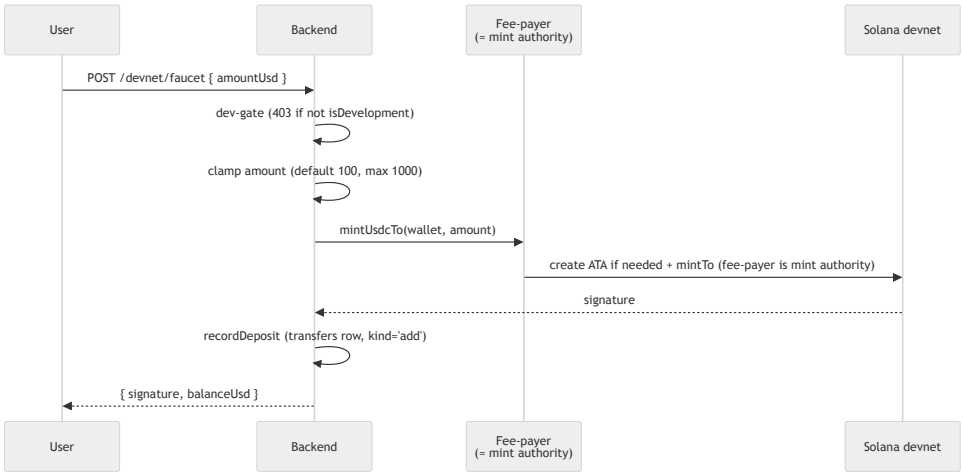
- `apps/backend/src/services/solana.ts` (`feePayer`, `buildUsdcTransfer`, `mintUsdcTo`)
- `apps/backend/src/services/escrowProgram.ts` (fee-payer = escrow authority)
- `apps/backend/src/config.ts` (`FEE_PAYER_SECRET_KEY`), `scripts/setup-devnet.ts` (generates it)
- Client signer: `apps/web/src/services/privy/PrivyBridge.tsx` (`signAndSend`)

04 · Money In (Funding the balance)

What it is

How dollars get into a user's balance. **Today this is a devnet faucet**: the backend *mints* controlled test-USDC straight into the user's wallet. In production this becomes a real **on-ramp** — the user adds money with a debit card / bank transfer / Apple Pay and a licensed partner converts fiat → USDC. The faucet is the single most "test-environment" part of the app and is **dev-gated**.

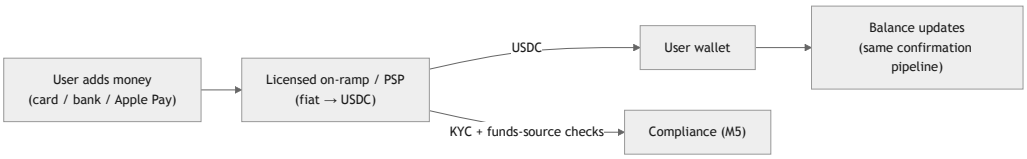
Flow — today (devnet faucet)



The backend's fee-payer is the **mint authority** of the controlled devnet USDC mint (created by `scripts/setup-devnet.ts`), so it can both pay the gas and mint the tokens — no external faucet, no rate-limited public airdrop. The route is hard **403** unless `config.isDevelopment`.

There's also a secondary **"deposit by address"** path in the web `AddMoney` sheet (show the wallet address, user sends USDC to it) — this is the most crypto-native surface left in the app and a candidate to hide/relabel for normal users.

Flow — future (real on-ramp, M7)



The internals downstream (balance read, transaction history, the confirmation pipeline) **don't change** — only the *source* of the USDC flips from "we mint it" to "a partner sells it to the user."

Why this way

- **Faucet-first keeps us building, not integrating.** A real on-ramp means a licensed partner, contracts, KYC, and settlement plumbing — none of which should block building the *product*. Minting test-USDC gives a realistic balance to exercise every other flow.
- **Reuse the fee-payer as mint authority.** One controlled mint we own means deterministic balances in dev and zero dependence on flaky public devnet airdrops.
- **Dev-gated, clamped.** `isDevelopment` + a 1000-cap means the faucet can never exist in prod or be abused for unbounded mint.

Legalities

- **On-ramp = the regulated edge.** Converting fiat to crypto for customers is exactly the activity that needs a **licensed partner** (a registered PSP / MSB / VASP), plus **KYC** and **source-of-funds** checks at the moment money enters. This is deliberately **deferred to M7**, and the faucet is explicitly **not** a money-movement feature — it mints valueless devnet tokens.
- **No real value today.** Because the mint is a controlled devnet token with no market value, the faucet moves nothing of legal consequence. The moment a real on-ramp lands, KYC (M5) must be enforced *before* funding, and the partner's licenses cover the fiat leg.

Status

- ✅ Devnet faucet (`/devnet/faucet`) — mints controlled USDC, records a deposit, returns new balance; dev-gated and clamped.
- 🟡 **"Add money" is still a faucet under the hood**, but the **de-crypto pass cleaned the copy** — *"Test dollars on devnet"* → *"Added to your balance instantly"*, *"Or deposit USDC"* → *"Or add manually"*. The user no longer sees the test/crypto nature; the *funding mechanism* is still a faucet.
- ⬜ **Real on-ramp** (card/bank, licensed partner, KYC-gated) — **M7**.

Key files

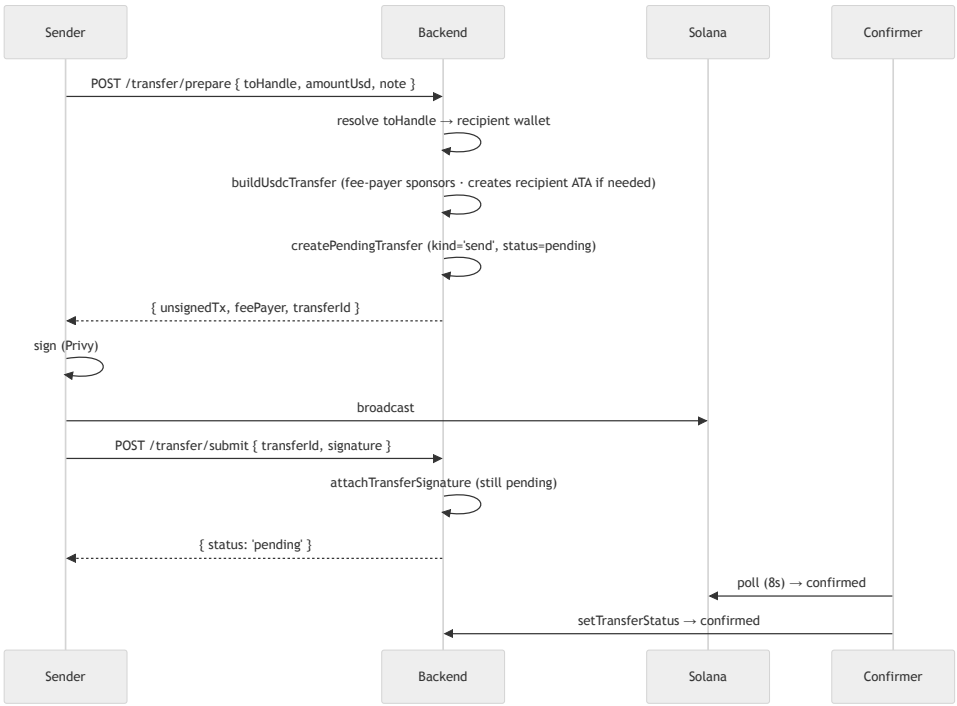
- Backend: `apps/backend/src/services/solana.ts` (`mintUsdcTo`), route `/devnet/faucet` in `routes/index.ts` , `db/queries.ts` (`recordDeposit`).
- Setup: `apps/backend/scripts/setup-devnet.ts` (creates the controlled mint, funds test users).
- Web: `src/components/sheets/AddMoney.tsx` , `app/onboarding/fund/page.tsx` .

05 · P2P Send & Request

What it is

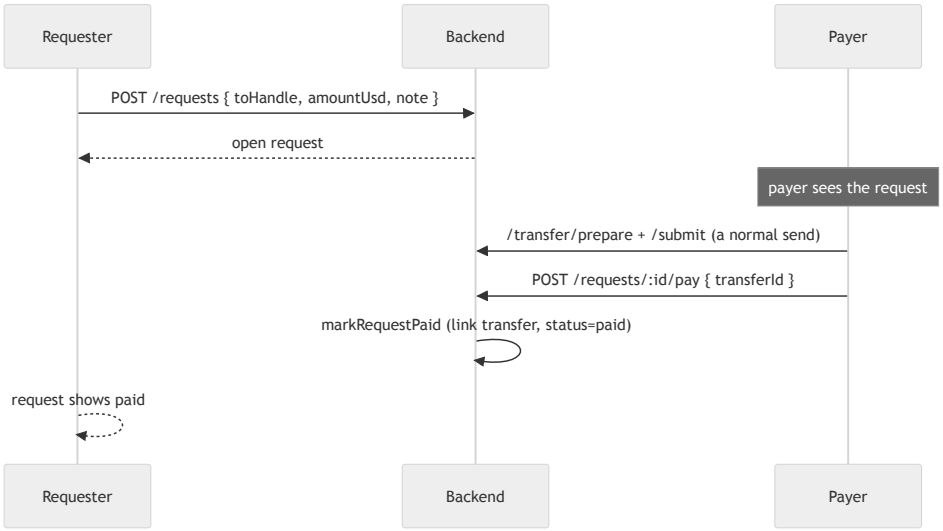
The Venmo/Cash App core: **send money to a friend by `@handle`**, and **request money** from someone. Sends are real on-chain USDC transfers, gas-sponsored so they feel instant and free. Requests are lightweight DB records that, when paid, link to the transfer that fulfilled them. This is the **M0 base wallet** — the oldest, most complete part of the app.

Flow — send money



The two-call shape (`prepare` then `submit`) is the universal seam from 01: the backend records the *intent* at prepare time so submit only needs the id + signature. Status starts `pending` and the confirmation pipeline flips it.

Flow — request money



A request is just a row in `payment_requests`. Paying it is an ordinary transfer; `/requests/:id/pay` links the fulfilling `transferId` and marks it `paid`. The requester can `/requests/:id/cancel` while it's still open.

Why this way

- **Handles + server-side resolution.** The sender types a name; the backend resolves it to a wallet and creates the recipient's token account (ATA) if it's their first time receiving — sponsored, so the recipient needs nothing set up in advance.
- **Record-intent-first.** Creating the pending transfer at prepare time means a dropped signature leaves a clean, expirable row (the confirmer retires stale unsigned transfers) rather than a lost payment.
- **Requests are not escrow.** A request is a *message* ("you owe me $X"), not held funds — so it's a trivial DB record, not an on-chain primitive. Only **merchant payouts** need escrow (07).

Legalities

- **P2P money transfer** between identified users is generally the *lightest* activity in the app, but it's still where **transfer limits and KYC tiers** belong (tier 1 caps small amounts; larger sends should require verification). These limits are now **enforced** (M5): tier 1 caps P2P at $100/tx plus a rolling daily cap, checked server-side with sanctions screening on the recipient.
- **AML / travel rule.** At real scale, P2P flows need transaction monitoring and (above thresholds) originator/beneficiary information sharing. Not in scope until real rails; the `transfers` ledger is the data foundation those checks will read.
- **Funds are the user's own.** Because it's self-custodial (03), a send is the user moving their own money — not the platform transmitting on their behalf — which is the lighter classification.

Status

- ✅ Send (`/transfer/prepare` + `/submit`), gas sponsorship, recipient ATA auto-creation, handle resolution, balance + history, payment requests (create/pay/cancel), confirmation via poller/webhook.
- ✅ **Mobile parity exists for P2P** (this is the part the Expo app *does* have; it lacks only the cash-out order flow).
- ✅ **Transfer limits enforced (M5)** — tier-1 caps P2P at $100/tx + a rolling daily cap, with sanctions screening on the recipient; over-limit returns a structured `limit_exceeded`.

Key files

- Backend routes: `/transfer/prepare`, `/transfer/submit`, `/requests*`, `/users/resolve` in `routes/index.ts`; `services/solana.ts` (`buildUsdcTransfer`); `db/queries.ts` (`createPendingTransfer`, `attachTransferSignature`, `createRequest`, `markRequestPaid`).
- Web: `src/components/sheets/friends.tsx`, `src/services/*`.
- Schema: `transfers`, `payment_requests` (see 10-data-model).

06 · Payout Order Lifecycle (Quote + State Machine)

What it is

The **cash-out / merchant-pay** flow — the heart of the off-ramp. The user wants to pay ₹X (or cash out to a payout rail like UPI). The app:

1. **Quotes** a locked rate + fee + ETA, valid **45 seconds** (the "certainty" the PDF demands).
2. On confirm, runs the payment through a **7-state order machine** designed **failure-first** — every step has a way to stall, escalate, and either recover or refund, so money is never lost or stuck.

This is the M1 milestone. Escrow (07) and routing (08) plug into it; this doc is the **spine**.

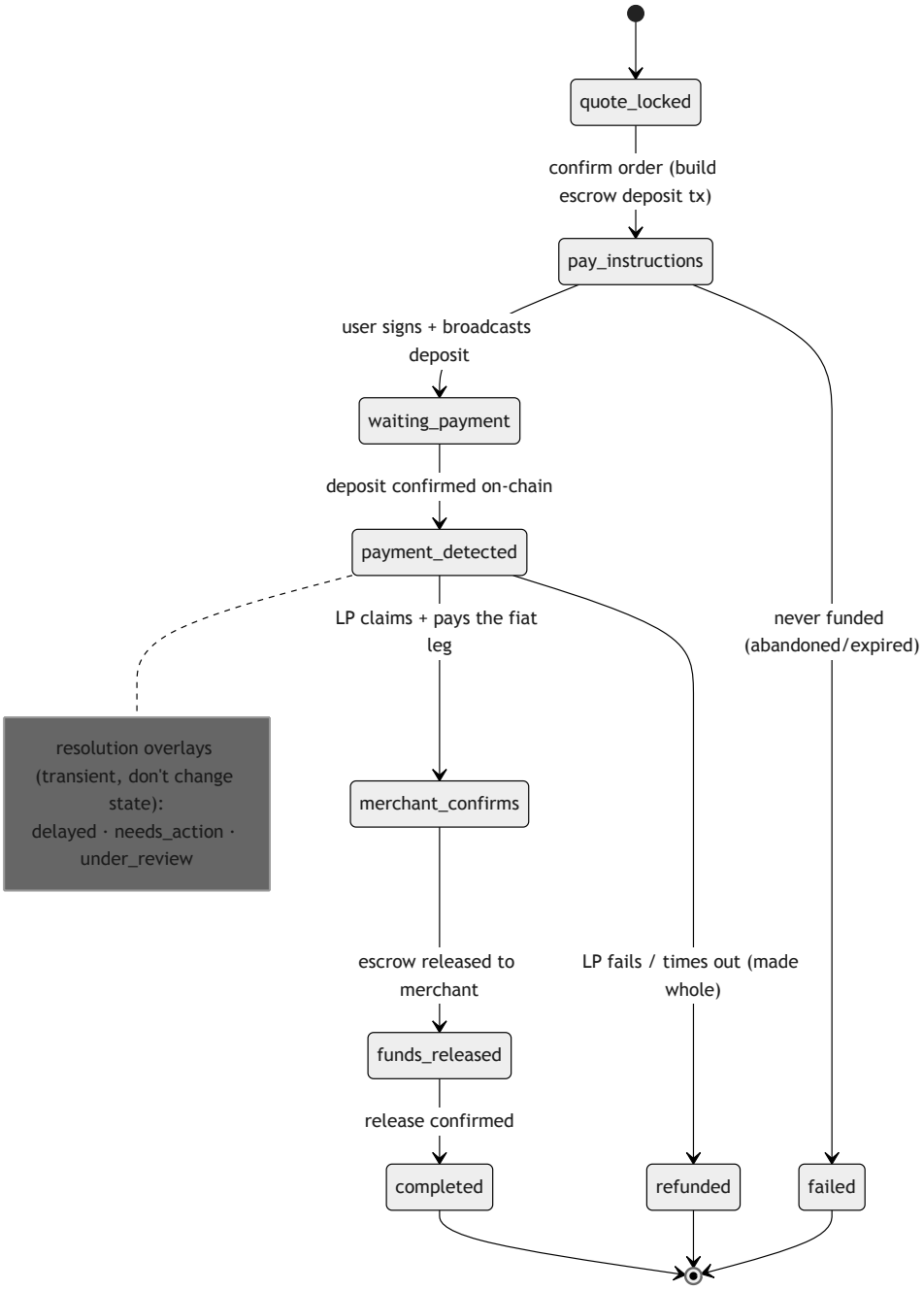
The quote

`quoteEngine.createQuote` picks a merchant (via routing), then prices:

```
rate      = getMidRate() × (1 − merchant.spread_bps / 10_000)   // worse-than-mid for the user
amountUsdc = amountInr / rate
fees      = flat platform fee + network fee (illustrative for M1)
eta       = merchant.avg_completion_secs … ×2
expiresAt = now + 45s
```

The quote is persisted (so the order can verify it hasn't expired) and carries `routeQuality`. The FX **mid rate is now live (M7)** — `getMidRate()` returns a cached, range-validated USD→INR rate refreshed in the background (with a safe fallback); the merchant spread is applied on top. See 14 for the infra layer.

Flow — the 7 states



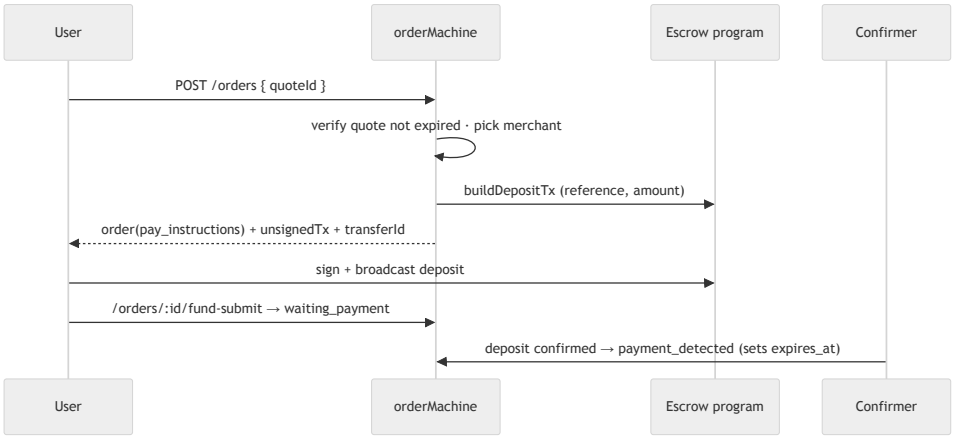
Happy path in words: lock a quote → confirm (the backend builds an escrow **deposit** tx) → user signs, USDC goes into the per-order **escrow** → the chain confirms → a merchant/LP picks it up and sends the fiat → the escrow is **released** to the merchant → done.

Failure-first design:

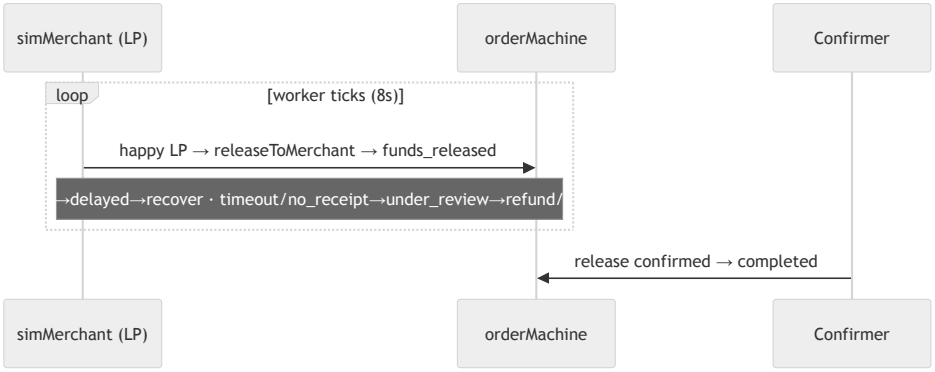
- A **resolution** overlay (`delayed` / `needs_action` / `under_review`) describes *what's happening* without leaving the state — so the timeline can say "taking a little longer" honestly.
- `expires_at` is an **absolute escalation deadline** for the current state; the worker escalates on it (not on unrelated `updated_at` bumps).
- If a merchant never delivers, the order **refunds** the user from escrow (terminal, user made whole) — or, with M3, **silently re-routes** to another merchant first.

Flow — confirm → fund → settle (sequence)

Place & fund



Settle (LP worker)



The order machine is the **only writer of order state**, and it uses **CAS (compare-and-set) transitions** (`casOrderState`) so the confirmer, the simulated worker, and dev endpoints can't race each other into an inconsistent state. Critically, **release/refund broadcast first, then transition** — a failed broadcast leaves the order in its actionable state for the worker to retry (no stuck orders).

Why this way

- **45s locked quote = "certainty."** The PDF's core promise is the user knows *exactly* what they'll pay before committing. The lock + expiry enforces it.
- **Failure-first states.** Off-ramps fail constantly (LP offline, wrong amount, slow bank). Modelling delayed/needs_action/under_review + a refund terminal up front means failures are *handled*, not surprises.
- **CAS + broadcast-before-transition.** Idempotent, race-safe transitions are what make the multi-actor lifecycle (user, confirmer, LP worker) trustworthy.

Legalities

- **Off-ramp / settlement.** Paying a merchant or cashing out to a bank rail is regulated **payout** activity — in production it needs a **licensed payout partner** (UPI PSP, etc.) and settles real fiat. Today the fiat leg is **simulated** (the "merchant" is a worker), so no real payout happens — the structure is real, the rail is not (M4/M7).
- **FX.** Quoting an INR↔USD rate and taking a spread has consumer-protection and (in some places) licensing implications. The rate is now a **live, cached, range-validated** feed (M7) and the quote screen itemizes the rate + fees; a regulated FX provider with an SLA is the remaining step.
- **Escrow / funds-in-flight.** While an order is mid-flight, the USDC sits in the on-chain escrow — see 07 for who controls it and why that's the key legal question.

Status

- ✅ Quote engine (rate/fee/ETA, 45s lock), full 7-state machine + resolution overlays, CAS transitions, escrow fund/release/refund wiring, the web Quote + OrderTracking UI, dev sim controls.
- ✅ **FX rate is live** (M7) — cached, range-validated USD→INR with a safe fallback; the spread applies on top. Fees are still illustrative; the **merchant/LP is simulated** (`simMerchant.ts`).
- ⬜ Real payout rails + mainnet (rest of M7).

Key files

- `apps/backend/src/services/quoteEngine.ts` , `orderMachine.ts` , `simMerchant.ts`
- Routes `/orders/quote` , `/orders` , `/orders/:id/fund-submit` , `/orders/:id` , `/orders/:id/cancel` , `/sim/*` in `routes/index.ts`
- `db/queries.ts` (order/quote/event queries, `casOrderState`), schema `quotes` / `orders` / `order_events`
- Web: `src/components/sheets/{Quote,OrderTracking}.tsx` , `src/services/orders.ts`

07 · On-chain Escrow (the Anchor program · M2)

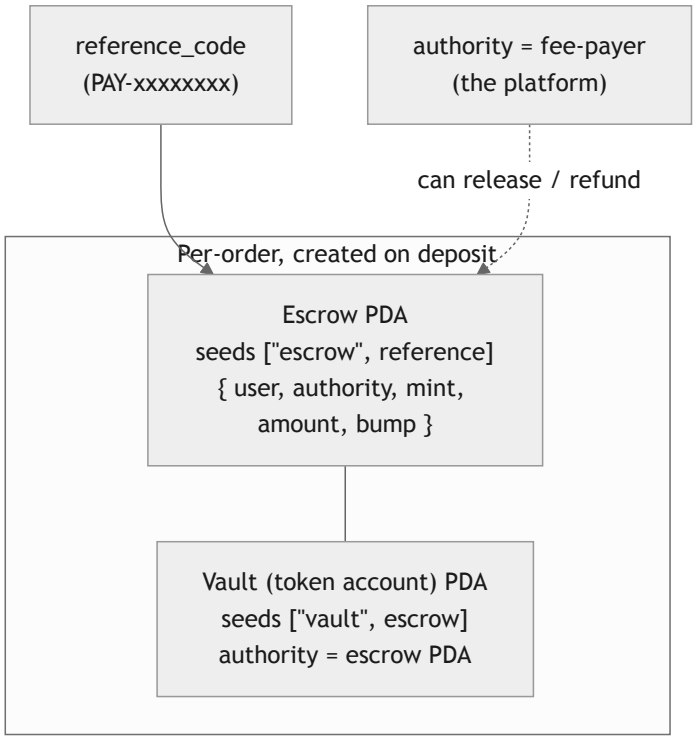
What it is

A **Solana program (smart contract)** that holds a user's USDC **while a merchant payout is in flight**, and releases it **only** to the merchant on success or **back to the user** on failure. Each order gets its **own escrow account** (a PDA) and a token **vault**, created when the user deposits and **closed** when the order resolves. Because the account is destroyed on release/refund, a **second release/refund is impossible** — the chain itself is the double-spend guard.

This replaced M1's "backend-custodied escrow" (funds held by a backend key). Now the rules live on-chain, not in our server.

- **Program id (devnet):** `9UfJyNduUw6hJexd7zbwc6ivqPLs2ie2edQrqYAYVv3i`
- **Anchor 0.32**, Rust; tested on localnet, deployed to devnet.

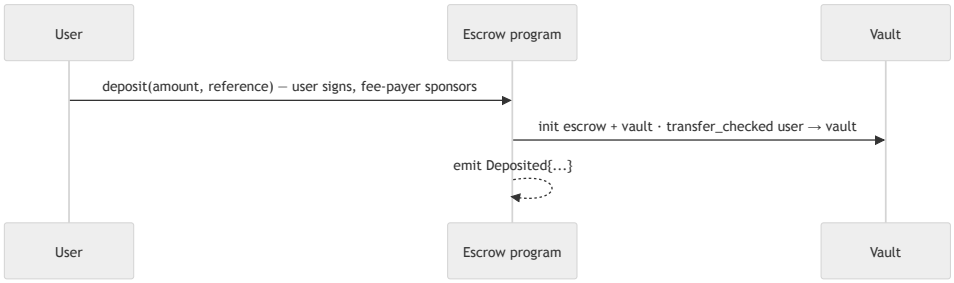
The accounts



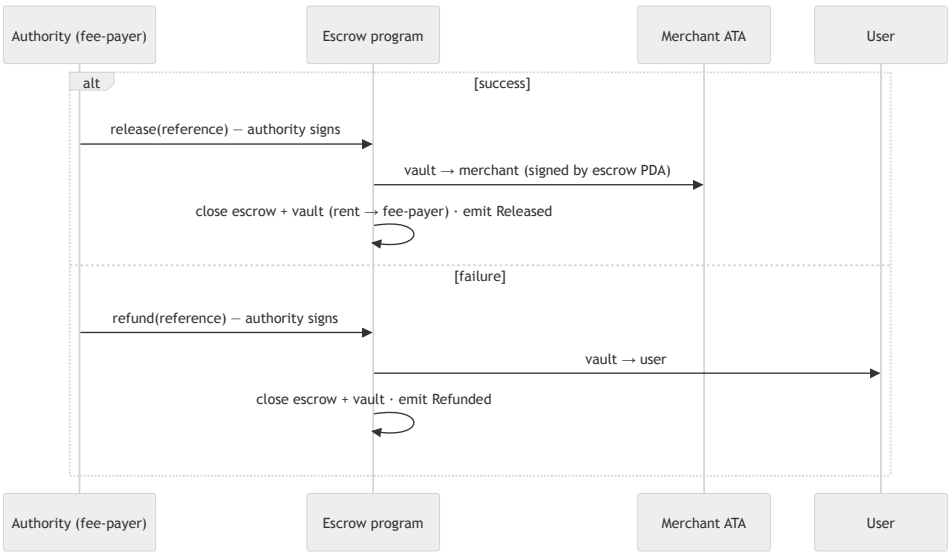
- **Escrow PDA** — `seeds = ["escrow", reference_code]`. Holds the metadata: who deposited, the authority allowed to release/refund, the mint, the amount, the bump.
- **Vault PDA** — `seeds = ["vault", escrow]`, a token account owned by the escrow PDA. The actual USDC sits here.
- **Authority** — set to the **fee-payer** (the platform) at deposit time; only it can release/refund (`has_one = authority`).

Flow — deposit → release / refund

Deposit



Release or refund



Three instructions:

- **`deposit(amount, reference)`** — user (token authority) + fee-payer (payer) sign. Inits escrow + vault, CPI `transfer_checked` user → vault, sets `authority = fee_payer`. Emits `Deposited`.
- **`release(reference)`** — authority signs; CPI vault → merchant ATA (signed by the escrow PDA's seeds), then **closes** escrow + vault (rent back to fee-payer). Emits `Released`.
- **`refund(reference)`** — authority signs; CPI vault → user, **closes** accounts. Emits `Refunded`.

The backend (`escrowProgram.ts`) builds these: `buildDepositTx` returns the same `{unsignedTx, feePayer}` shape as a normal transfer (so the client signer is unchanged), while `releaseEscrow` / `refundEscrow` are authority-signed and broadcast server-side. The IDL is **vendored** into `apps/backend/src/idl/escrow.json` (since `anchor/target` is gitignored).

Why this way

- **Per-order account, closed on terminal.** This is the elegant part: a closed account *cannot* be released again, so **double-release and release-vs-refund are impossible on-chain** — no backend lock needed, the strongest possible guard.
- **`has_one = authority`.** Only the platform can move escrowed funds to a merchant or back — but it can only ever do one of those two, and never redirect funds elsewhere (the destination is checked).
- **`emit!` events.** Deposited/Released/Refunded are an on-chain **audit log** (the PDF's "Layer E" proofs) — independently verifiable, not just our database.
- **Same client seam.** Deposit reuses prepare→sign→submit, so moving escrow on-chain cost ~zero frontend change.

Legalities

- **Funds-in-flight custody.** This is the one place value is under program control. The design is **trust-minimized conditional settlement**: the user authorizes the deposit, and the program can only (a) pay the intended merchant or (b) return the funds to

the user — it can't divert them. That's a meaningfully different posture from "the platform holds customer balances."

- **The key question = the release authority.** Today the **fee-payer (platform) holds the release authority**, which means *operationally* we decide when funds release. For a fully trust-minimized off-ramp, that authority should be constrained further (e.g. conditioned on a verifiable payout proof, time-locked auto-refund, or multisig). That hardening is future work and is the main legal gray for this flow — flagged in 11.
- **Auditability.** The on-chain events + closed-account guarantee make disputes and reconciliation provable, which is a compliance asset.

Status

- ✅ Program built, **4/4 localnet tests pass** (deposit→release, deposit→refund, double-release rejected, non-authority release rejected), **deployed to devnet**, backend wired, IDL vendored, typecheck clean.
- 🟡 **Interactive web E2E pending a human** (sign a real `deposit` from a funded wallet, watch the escrow PDA close on completion). Authority model is platform-held (see Legalities).
- ⬜ Registry/staking, pause/circuit-breaker, constrained release authority — later milestones.

Key files

- Program: `anchor/programs/escrow/src/lib.rs`, tests `anchor/tests/escrow.ts`, `Anchor.toml`
- Backend: `apps/backend/src/services/escrowProgram.ts`, `idl/escrow.json`, config `ESCROW_PROGRAM_ID`
- Build/deploy recipe (offline workaround): see the repo memory `anchor-build-offline`.

08 · Routing, Scoring & the LP Marketplace (M3 + M4)

> The PDF calls this the **moat**: *"a marketplace, not a DEX."* It's **100% invisible to the user** — per the product decision *"users don't care about routes, just pay like UPI."* No route picker, no labels; the engine makes better choices, learns from outcomes, and silently recovers.

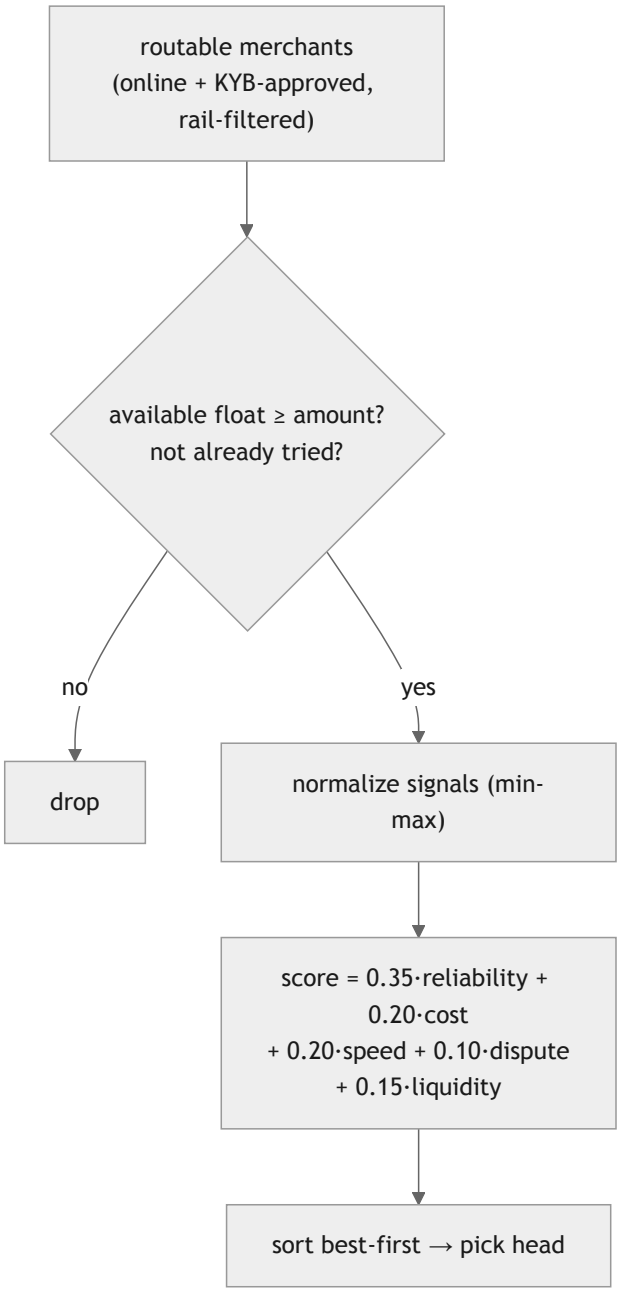
What it is

Three backend-only capabilities over the merchant/LP network:

1. **Multi-signal ranking (M3)** — score every eligible liquidity provider on reliability, cost, speed, dispute rate, and liquidity, and pick the best one.
2. **Silent auto-failover (M3)** — if the chosen LP stalls past its SLA, **re-route** the order to the next-best LP and retry the payout, instead of refunding. Refund only when no candidate is left.
3. **The merchant/liquidity layer (M4)** — the LP network behaves like a real marketplace: signals are **computed from real order outcomes** (not static seed numbers), **float is tracked live**, LPs are gated by **KYB + lifecycle**, and **bonds + disputes** give accountability. A flaky LP's score drops and it gets out-ranked/suspended automatically.

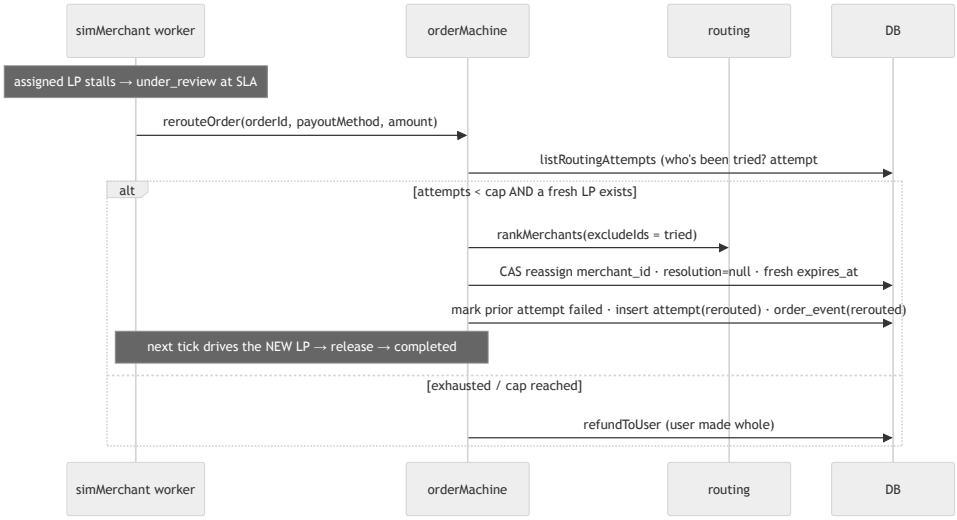
The unlock that makes failover cheap & safe: the **escrow holds USDC independent of merchant** (07) — the on-chain deposit names no merchant; only *release* picks a treasury. So a reroute is a pure **DB + worker** change (swap `orders.merchant_id`, retry release to a new treasury) — **no second deposit, no Anchor change, escrow stays funded.**

Flow — rank a merchant (at quote time)



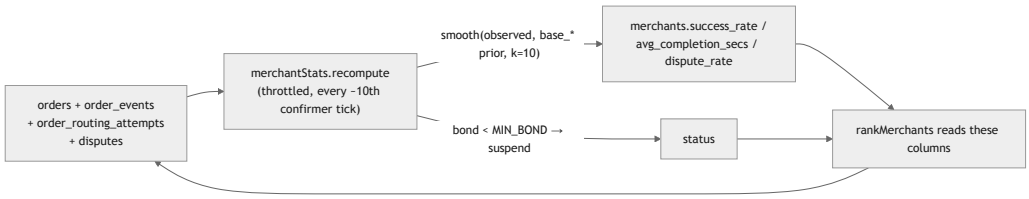
Reliability-weighted (matching "expose certainty"). **Available float** = `float_usdc − committed` (sum of in-flight orders), so a saturated LP drops out until it clears (this folded in the old M3 queue-depth signal). `quoteEngine` uses the top-ranked LP; `routeQuality` derives from the composite score (internal — not shown).

Flow — silent failover



`MAX_ROUTING_ATTEMPTS = 3` (initial + 2 reroutes). Every assignment/reroute/release/failure is recorded in **`order_routing_attempts`** — the "already tried" set, the observability log, **and** the substrate M4 scores on. The user's timeline just keeps advancing; the reroute is logged, not shown.

Flow — the M4 learning loop (signals stay live)



`merchantStats.recomputeMerchantStats()` derives each LP's live signals from history — `success_rate` from released-vs-failed attempts, `avg_completion_secs` from `fund_confirmed → funds_released` event timestamps, `dispute_rate` from upheld disputes — each **smoothed toward the seed `base_*` prior** (Bayesian, k≈10) so a low-volume LP isn't volatile. Runs on a throttled confirmer tick. Because ranking already reads those columns, routing self-corrects with **zero ranking-code change**.

The merchant lifecycle (M4)

- **Onboarding / KYB** — `kyb_status` (`pending → approved/rejected`). A prospective LP applies via `POST /merchants/apply` (created `pending` /offline); a dev `POST /sim/merchants/:id/kyb` approves it (flips online). Only **online + approved** LPs are routable (`listRoutableMerchants`).
- **Float / inventory** — `float_usdc` is the LP's capacity; **committed** float (in-flight orders) is subtracted live, so the marketplace never over-allocates an LP.
- **Bonds** — `bond_usdc` is skin-in-the-game; the recompute pass **auto-suspends** any LP below `MIN_BOND` (2000).
- **Disputes** — a user opens one on a completed order (`POST /orders/:id/dispute`); a dev resolution (`/sim/disputes/:id/resolve?for=user|merchant`) **upheld** for the user **draws down the LP's bond** (1500 fine) and feeds `dispute_rate` — so bad behaviour costs the LP money *and* routing share.

Why this way

- **Invisible by design.** A stalled merchant becomes the platform's problem, not the user's: an order that would have **refunded** now silently **completes**.
- **Signals that learn.** M3 ranked on static seed numbers; M4 makes them reflect reality, so the marketplace gets better (or drops bad LPs) on its own — the whole point of "a marketplace."

- **Accountability without new custody.** Bonds + disputes punish bad LPs economically; failover and reroute never create a second deposit, so the custody posture from 07 is unchanged.

Legalities

- **Best-execution / fair routing.** The `order_routing_attempts` log + explicit scoring weights are the evidence trail that routing isn't self-dealing — exactly what an auditor would want.
- **Merchant KYB + bonds + disputes** are modelled (simulated) here; a real network needs real KYB, contracts, and a real dispute/arbitration process with a licensed operator (see 11-compliance). The **structure** is in place; the **vendors** are deferred to M7.
- **No new custody surface** — reroute/release just target a different vetted treasury.

Status

- ✅ **M3 shipped** — weighted `rankMerchants`, `dispute_rate` + `order_routing_attempts`, `rerouteOrder`, worker reroute-before-refund, 5 seeded LP profiles, dev `merchant-fail` trigger. Frontend untouched.
- ✅ **M4 shipped** — live available-float gating, `merchantStats` computed+smoothed scoring on the confirmer tick, KYB lifecycle + onboarding, bonds + auto-suspend, disputes flow. `yarn typecheck` clean; a self-fixturing `check-merchant.ts` harness asserts the learning/float/suspend behaviour.
- 🟡 The LP network is still **simulated** (no real KYB vendor / payout / arbitration) — flips at M7.

Key files

- `apps/backend/src/services/routing.ts` (`rankMerchants`, available float), `quoteEngine.ts`, `orderMachine.ts` (`rerouteOrder`), `simMerchant.ts` (reroute-before-refund), **`merchantStats.ts`** (`recomputeMerchantStats`, `MIN_BOND`, bond/suspend).
- `db/schema.sql` (`merchants` + `kyb_status` / `bond_usdc` / `base_*`, `order_routing_attempts`, `disputes`), `db/seed.ts` (5 LP profiles + priors), `db/queries.ts` (ranking, float, stats, lifecycle, disputes).
- Routes in `routes/index.ts`: `/merchants/apply`, `/orders/:id/dispute`, dev `/sim/orders/:id/merchant-fail`, `/sim/merchants/:id/kyb`, `/sim/merchants/recompute`, `/sim/disputes/:id/resolve`.
- Harnesses: `scripts/check-routing.ts`, `scripts/check-merchant.ts`.

09 · Confirmation Pipeline

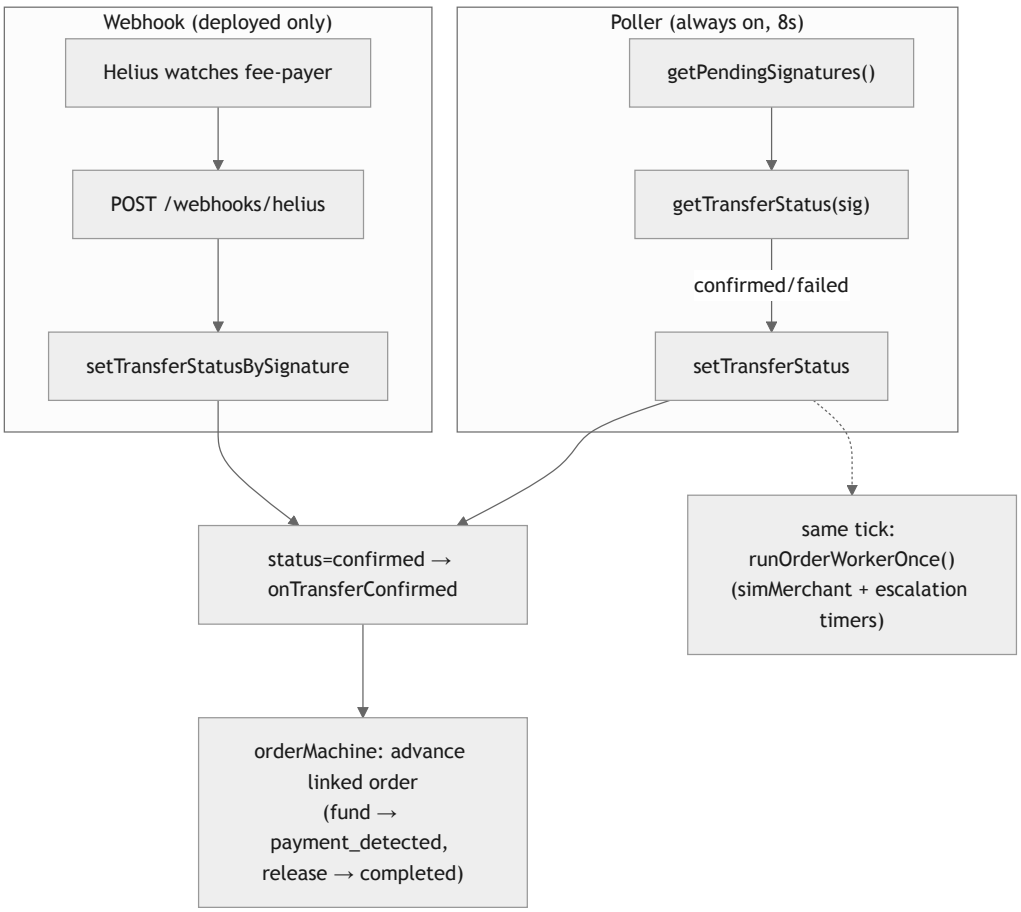
What it is

How the backend learns that an on-chain action actually landed. Solana confirmation is **asynchronous** — after a transaction is broadcast, something has to watch the chain and flip the database row from `pending` to `confirmed` / `failed`, then advance any order that was waiting on it. Two mechanisms do this, by design:

1. **Confirmer poller** (`confirmer.ts`) — every **8 seconds**, resolves every pending+broadcast transfer and persists a terminal status. Always on. The reliable fallback.
2. **Helius webhook** (`helius.ts` + `/webhooks/helius`) — Helius *pushes* a confirmation the moment a watched transaction lands. Faster, but only active on a deployed backend (needs `HELIUS_API_KEY` + `PUBLIC_BASE_URL`).

Both write through the **same** function, so they're interchangeable and idempotent.

Flow



Why watching the fee-payer catches everything: the fee-payer **signs every sponsored transaction** (sends, escrow deposits/releases, faucet mints), so a single Helius subscription on the fee-payer address sees all of them — no per-user subscriptions needed.

The poller does double duty. Each 8s tick also calls `runOrderWorkerOnce()` (the simulated merchant + order-escalation clock) and retires stale unsigned transfers. So the confirmer is both the chain-watcher *and* the heartbeat for the order lifecycle.

Order advancement. When a transfer flips to `confirmed` , `onTransferConfirmed(transferId)` looks up the linked order and advances it: an `escrow_fund` confirmation → `payment_detected` (and sets the SLA `expires_at`); an `escrow_release` confirmation

→ `completed`. A confirmed P2P **send** also fires an in-app notification to the recipient ("Neha sent you $5" — M6, see 14).

Why this way

- **Webhook + poll, not one or the other.** The webhook is fast but depends on external infra and a public URL; the poller is slower but unconditional. Running both means **push speed in prod, guaranteed convergence everywhere** — and because both are idempotent, a double-delivery is harmless.
- **Keyed off the signature row.** Confirmation logic only needs `transfers.signature` + status, so it's fully decoupled from *what* the transaction did (send vs. escrow vs. faucet). New transaction types plug in with zero confirmer change.
- **One process, one clock.** Folding the order-escalation worker into the same tick avoids a separate queue/cron — and the tick now also recomputes merchant stats + sweeps the rate-limit windows (14). A Redis-backed queue (multi-instance) is the remaining **M7** scale step.

Legalities

- **Settlement finality & reconciliation.** A payments system must know, provably, whether money moved. The pipeline ties every DB row to an on-chain signature with a terminal status — the foundation for **audit, dispute resolution, and reconciliation**. Combined with the escrow's on-chain `emit!` events (07), there's an independently verifiable record of settlement.
- **Operational resilience.** "We lost track of a payment" is both a UX and a compliance failure. The redundant confirm-and-retry design (plus stale-transfer expiry) is what prevents stuck/lost states.

Status

- ✅ Confirmer poller (8s) with order advancement + worker tick + stale-transfer expiry; Helius webhook registration (idempotent) and handler with optional shared-secret auth.
- 🟡 Webhook is **only active when deployed** (needs `HELIUS_API_KEY` + `PUBLIC_BASE_URL`); locally the poller carries confirmation. No Redis-backed retry/backoff queue yet (multi-instance → **M7**).

Key files

- `apps/backend/src/services/confirmer.ts` (`confirmPendingOnce`, `startConfirmer`)
- `apps/backend/src/services/helius.ts` (webhook registration), route `/webhooks/helius` in `routes/index.ts`
- `apps/backend/src/services/solana.ts` (`getTransferStatus`), `orderMachine.ts` (`onTransferConfirmed`)
- `db/queries.ts` (`getPendingSignatures`, `setTransferStatus[BySignature]`, `expireStaleUnsignedTransfers`)

10 · Data Model

What it is

The Postgres schema — the system's source of truth for everything *except* token balances (those live on-chain and are read live via `getUsdcBalance`). There's **no ORM**; one SQL file defines the schema and one module (`queries.ts`) holds every query. **Access control is enforced in the API layer** (no row-level security) — every query is owner-scoped by `userId`.

Entity relationships

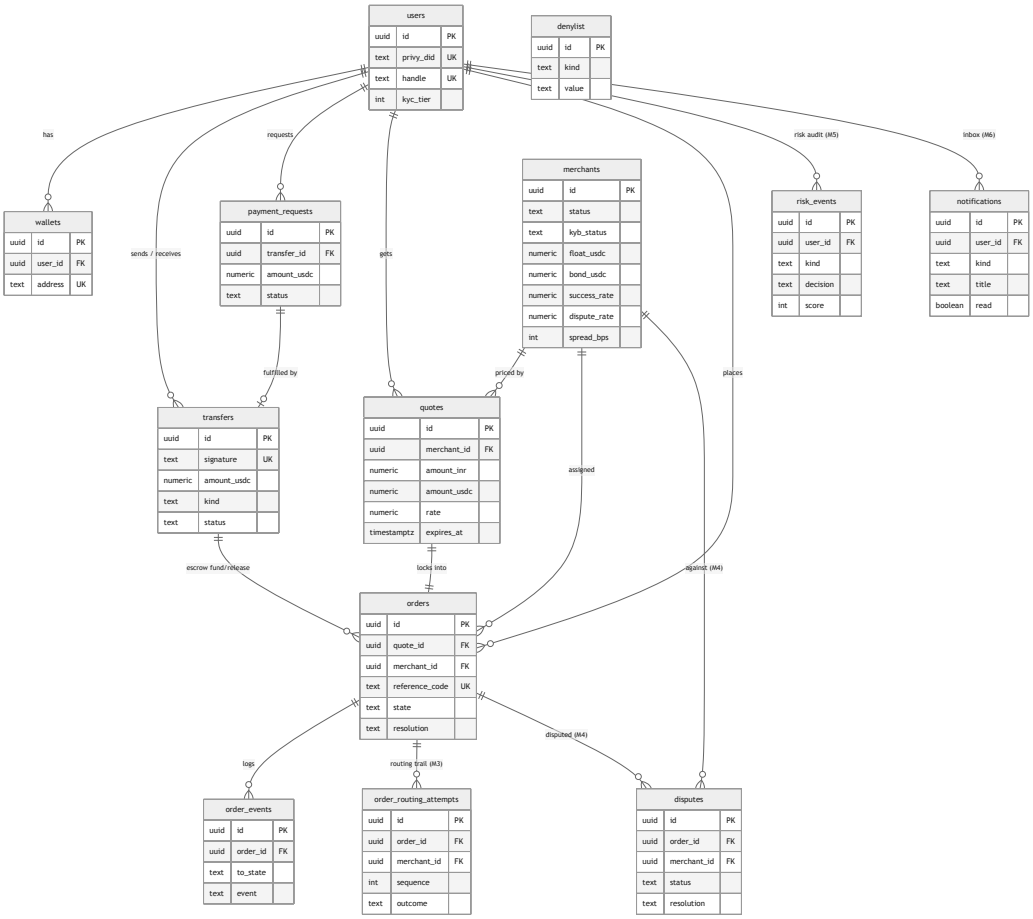


Table by table

Table	Purpose	Notes
users	one row per person	keyed to Privy DID; unique `handle`; `kyc_tier` drives M5 limits
wallets	a user's chain address(es)	`chain='solana'`; unique `address`
transfers	the money ledger	`kind ∈ send/add/pay/escrow_fund/escrow_release`; `status ∈ pending/confirmed/failed`; unique `signature`
payment_requests	"you owe me $X"	`status ∈ open/paid/cancelled`; links the fulfilling `transfer_id`
merchants	the LP network (simulated)	ranking signals (`success_rate` / `dispute_rate` / `avg_completion_secs` / `spread_bps` / `float_usdc`) kept **live by M4** + `base_*` seed priors; lifecycle (`status` incl. `suspended`, `kyb_status`), `bond_usdc`, `sim_behavior`
quotes	a 45s-locked price	INR + USDC legs, `rate`, fees, ETA, `route_quality`, `expires_at`
orders	the 7-state machine	`state` + transient `resolution`; links escrow fund/release transfers; `expires_at` = escalation deadline
order_events	append-only timeline	powers the live tracker; `(from_state, to_state, event, detail)`
order_routing_attempts *(M3)*	routing audit + tried-set + **M4 scoring substrate**	`(sequence, outcome, reason)`; `released` / `failed` outcomes feed `success_rate`
disputes *(M4)*	user disputes on completed orders	`(status open/resolved, resolution upheld/rejected)`; upheld draws down the LP's bond + feeds `dispute_rate`
denylist *(M5)*	simulated sanctions list	`(kind address/handle, value)`; screened on every money movement
risk_events *(M5)*	per-action fraud/screening audit	`(kind, decision allow/review/block, score, reason)`
notifications *(M6)*	in-app inbox	`(kind, title, body, read, meta)`; written by `notify()` at lifecycle events

Why this way

- **Ledger + state machine, not balances.** Token balances are authoritative on-chain; the DB records *intent and lifecycle* (who tried to pay whom, what state an order is in). This avoids the classic bug of a DB balance drifting from the chain — we read the chain for truth and reconcile.
- **Append-only `order_events`.** The timeline is derived from an immutable log, so the UI and any audit see the same history; state is never "rewritten."
- **CHECK constraints + idempotent migrations.** States/kinds are constrained at the DB level; a bottom-of-file `do $$ … $$` block backfills constraints on older databases (e.g. widening `transfers.kind` for escrow). No migration framework yet — `schema.sql` is re-runnable.
- **Indexes** cover the hot reads: transfers by user, orders by user and by `(state, expires_at)` (the worker's scan), events by order.

Legalities

- **The audit substrate.** `transfers`, `orders`, `order_events`, and `order_routing_attempts` together are the **record-keeping** any payments/AML regime requires: who moved what, when, through which provider, with what outcome. Combined with on-chain signatures + escrow events, it's a complete, cross-verifiable trail.
- **PII & retention.** `users` holds email/phone — a real launch needs retention limits, encryption at rest, and regional data rules (DPDP/GDPR). Minimal today.
- **No RLS.** Authorization is entirely in the API layer. That's acceptable now but a production hardening item (defense in depth).

Status

- ✅ Full schema through M6: Phase-1 + orders/quotes/merchants (M1), escrow transfers (M2), `order_routing_attempts` (M3), `disputes` + merchant lifecycle columns (M4), `users.kyc_tier` + `denylist` + `risk_events` (M5), `notifications` (M6); idempotent constraint + column backfills; owner-scoped queries.
- ⬜ RLS, encryption-at-rest, retention policy (production hardening).

Key files

- `apps/backend/src/db/schema.sql` (the whole schema), `db/queries.ts` (every query), `db/seed.ts` (merchants + test data), `db/pool.ts` (connection).
- Init/seed: `yarn workspace @pay/backend db:init` / `db:seed`.

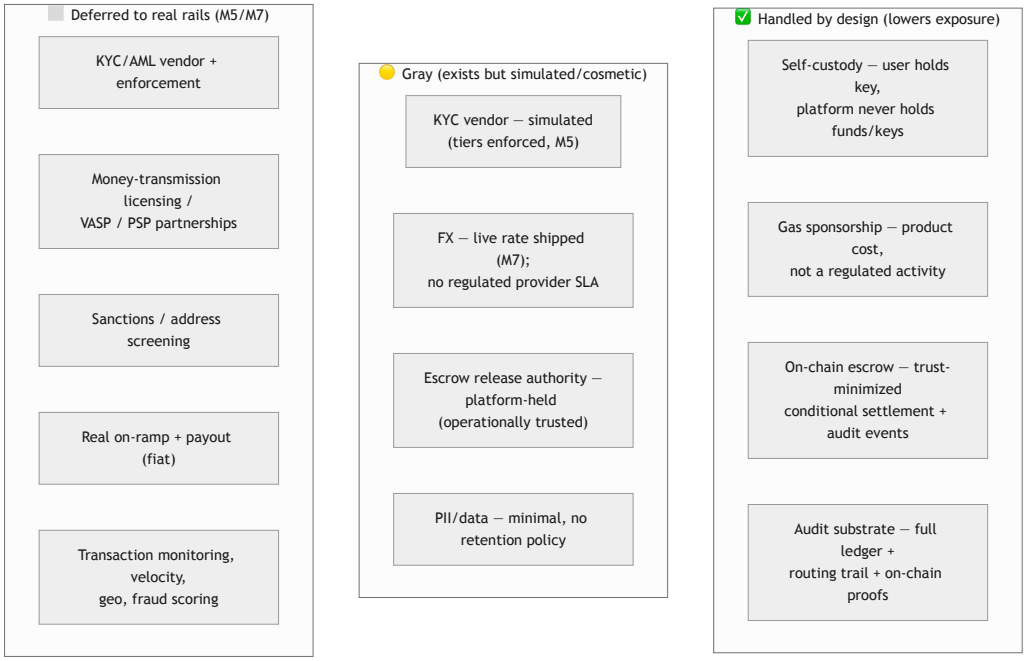
11 · Compliance & Legalities (the honest risk map)

> **Not legal advice.** This is an engineering-level map of *which regulatory questions each part of the product touches, how the current design handles or defers them, and where the gray zones are.* Every item marked for a real launch must be confirmed with qualified counsel in each target jurisdiction.

The core thesis: simulated-rails-first keeps us out of scope

The product is built **structure-first with simulated rails**: real on-chain escrow and a real order lifecycle, but a **simulated merchant/LP network**, a **devnet faucet instead of a real on-ramp**, and **no real fiat payout** (live FX + enforced KYC tiers have since shipped). Because **no real customer money and no real fiat movement occur**, almost none of the heavy regulation is *triggered yet*. The design intent is that each regulated edge has a **clean seam** where a licensed partner or vendor plugs in at the milestone that needs it — so we build the whole machine now and turn on the regulated parts last (M5 compliance, M7 real rails).

The risk map



Topic by topic

1. Custody & money transmission — ✅ *lightened by design*

Holding customer funds/keys is what triggers the heaviest rules (US state **MTL** / FinCEN **MSB**, EU **VASP/EMI**). The app is **non-custodial**: the user signs every movement of their own funds; the platform only sponsors gas. This is the **single biggest lever** keeping us in a lighter classification during the build. **Caveat:** the per-order escrow does put funds under *program* control briefly — see #3. *(The custody/MTL line is fact-specific and jurisdiction-specific; confirm with counsel before real funds move.)*

2. KYC / customer identification — ✅ *enforced (M5); real vendor at M7*

The tiered model is now **enforced**: `users.kyc_tier` gates money movement (tier 1 P2P ≤ $100/tx; tier 2 unlocks merchant pay), checked server-side against rolling-24h volume, with a simulated verification upgrade. **Still simulated** for the vendor leg — no real document collection / KYC-AML provider (Persona/Onfido/Sumsub land at **M7**).

3. Escrow release authority — 🟡 *platform-held*

The escrow is trust-minimized (funds can only go to the intended merchant or back to the user), but the **release authority is the platform's fee-payer** — so *operationally* we decide when to release. A fully trust-minimized off-ramp constrains this further (payout-proof-conditioned release, time-locked auto-refund, or multisig). This is the main legal gray of the payout flow. See 07.

4. FX & consumer disclosure — ✅ *live rate (M7); spread disclosure good*

Quoting INR↔USD with a spread has consumer-protection (and sometimes licensing) implications. The rate is now a **live, cached, range-validated** USD→INR feed (M7) and the quote screen itemizes rate + fees, so the disclosure is accurate. Remaining for a real launch: a regulated FX provider with an SLA + the formal disclosures that pair with real payout (**M7 vendor leg**).

5. On-ramp / payout (real fiat) — ⬜ *deferred to M7*

Converting fiat↔crypto and paying out to bank rails (UPI/Pix/bank) is the **regulated core** and needs **licensed partners** (PSP/MSB/VASP) who carry the fiat-leg licenses, plus KYC + source-of-funds at entry. Deliberately last; the faucet/simulated payout move nothing of legal consequence until then.

6. AML / sanctions / monitoring — 🟡 *structure built (M5); real lists/geo at M7*

Sanctions/address **screening** (against a seeded denylist), a per-tx **fraud score** (allow / review / block), and rolling-volume monitoring are **built and wired** (M5), writing a `risk_events` audit trail. **Simulated**: the denylist is seeded (not a live OFAC/sanctions feed) and geo checks are deferred (need IP). Real lists + a monitoring vendor land at **M7**.

7. Best-execution / fair routing — ✅ *evidenced*

A routing marketplace should prove it isn't self-dealing. The `order_routing_attempts` log + explicit scoring weights (08) are the evidence trail. (Not user-facing by choice.)

8. Data / privacy — 🟡 *minimal*

`users` holds email/phone/handle. A real launch needs a privacy policy, retention limits, encryption at rest, and regional handling (India **DPDP**, **GDPR**). Minimal today.

9. Merchant/LP network (KYB) — 🟡 *modelled (M4 ✅), real vendor deferred to M7*

The **structure** is now built (08): KYB lifecycle (`kyb_status` , onboarding/approval gating), performance scoring from real outcomes, **bonds** (skin-in-the-game, auto-suspend below a floor), and a **dispute** open/resolve flow that draws the bond down. Still **simulated** — no real KYB vendor, contracts, payout, or arbitration — so no real counterparty risk yet; those flip at **M7**.

Summary table

Topic	Posture	Status	Closes at
Custody / money transmission	non-custodial (lighter class)	✅ by design	(revisit M7 w/ counsel)
Gas sponsorship	product cost	✅	rate-limit M6
Escrow conditional settlement	trust-minimized	✅ structure / 🟡 authority	hardening later
KYC tiers	enforced per-tier caps (sim vendor)	✅ M5 / 🟡 vendor	vendor **M7**
AML / sanctions / monitoring	screening + fraud score built (sim)	🟡 M5	real lists **M7**
FX & disclosure	**live cached rate** + itemized spread	✅ M7	provider SLA later
On-ramp / payout (fiat)	simulated	⬜	**M7**
Merchant KYB / bonds / disputes	structure built (M4 ✅), simulated network	🟡	real vendor **M7**
Data / privacy	minimal	🟡	pre-launch

Bottom line

The app is intentionally on the **light side of regulation** today because it holds no customer funds and moves no real fiat. **KYC enforcement + sanctions screening are now built (M5, simulated vendor);** the remaining **gray** items (FX, escrow authority, PII) are known and seamed, and the **deferred** items (licensing, real rails, real KYC/AML + sanctions feeds) are sequenced into **M7** where a licensed partner carries the regulated leg. The thing to *not* do: flip on a real on-ramp or real payout before the real KYC/AML vendor + sanctions lists + counsel sign-off.

Related

03 Custody & gas · 07 Escrow · 12 Status & roadmap · 13 Future milestones

12 · Status & Roadmap (M0 → M7)

The heat-map

How done is each layer of the north-star architecture. ✅ complete · 🟡 gray (partial/simulated) · ⬜ planned.



Layer-by-layer

Architecture layer	Today	Status	Milestone
Auth & embedded wallet	Privy email-OTP, self-custody	✅	M0
Custody & gas sponsorship	fee-payer model, no SOL for users	✅	M0
P2P send / request	real gas-sponsored USDC + requests	✅	M0
Money in (funding)	**devnet faucet**	🟡	M0 → M7
Quote engine	rate+fee+ETA, 45s lock; **live FX** (M7)	✅	M1 / M7 FX
Order state machine	7 states + resolutions, CAS, failure-first	✅	M1
Escrow / conditional release	**on-chain Anchor PDA** (devnet)	✅	M2
Routing / matching engine	weighted rank + **silent failover**	✅	M3
Merchant / liquidity layer	live float, **computed scoring**, KYB, bonds, disputes (simulated network)	✅ / 🟡 sim	M4
Compliance & risk (KYC/AML)	**enforced KYC tiers** + screening + fraud score (sim vendor)	✅ / 🟡 vendor	M5
Notifications, rate-limit, observability	**in-app inbox + advisory lock + in-house rate-limit + /metrics** (single-instance)	✅ / 🟡 scale	M6
Real rails (live FX, payout, KYC vendor, mainnet)	simulated	⬜	M7

What "done" means per shipped milestone

- **M0 — Base P2P wallet** ✅ — Privy auth + embedded wallets, self-custody + gas sponsorship, send / request by handle, balance + history, devnet faucet, confirmation pipeline (poller + Helius webhook).
- **M1 — Quote + order lifecycle** ✅ — quote engine (45s lock), the full 7-state order machine + 4 resolution states, web Quote + OrderTracking UI, simulated merchant worker.
- **M2 — On-chain escrow** ✅ — per-order Anchor PDA (deposit/release/refund, close-on-terminal double-spend guard, `emit!` events), deployed to devnet, 4/4 localnet tests. *Gray: interactive web E2E pending a human; release authority platform-held.*
- **M3 — Routing + auto-failover** ✅ — weighted multi-signal ranking + invisible reroute-before-refund
  - `order_routing_attempts` audit log + dev `merchant-fail` trigger. Backend-only.
- **M4 — Merchant/liquidity layer** ✅ — live available-float gating, computed+smoothed performance scoring on the confirmer tick, KYB lifecycle + onboarding sim, bond reserve + auto-suspend, disputes open/resolve flow. `check-merchant.ts` harness. *Gray: the LP network itself is still simulated.*

- **M5 — Compliance & Risk** ✅ — `users.kyc_tier` enforced per-tier caps at the money-movement endpoints (rolling-24h volume, structured `limit_exceeded`), simulated verification upgrade, sanctions/denylist screening + a per-tx fraud score → `risk_events`. The web uses the real tier. `check-compliance.ts` harness. *Gray: KYC vendor + real sanctions lists still simulated (M7).*
- **M6 — Notifications + infra** ✅ — in-app notifications (table + emits at order completed/refunded, send→recipient, request paid, KYC upgrade + web bell/badge/inbox), a per-user **advisory lock** closing the M5 daily-cap TOCTOU, an in-house **rate limiter** (money-movement + faucet + auth), and **metrics** at `/metrics` + a pino latency log. `check-infra.ts` 7/7. *Gray: single-instance — Redis + real push are M7. See 14.*
- **De-crypto UX** ✅ — dollars-only Quote, human payment tracker, Processing/Completed chips, copy swaps; the app reads like an everyday money app, no crypto "tells" user-facing.

The honest gray list (what's not real yet)

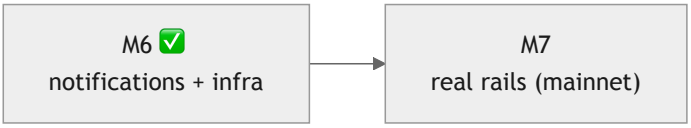
1. **FX is live** now (cached, range-validated, with an offline fallback); the remaining gray is a *regulated* FX provider with an SLA + formal disclosures, bundled with real payout. *(M7 vendor)*
2. **The LP network is simulated** — `sim_behavior` drives outcomes; KYB/bonds/disputes are modelled, not backed by a real vendor or arbitration. *(M7)* But the signals they feed are now **real/computed**.
3. **Funding is a faucet**, not a real on-ramp. *(M7)*
4. **KYC vendor + sanctions lists are simulated** — tiers are now **enforced** (M5), but real document capture and a live OFAC/sanctions feed are sim. *(M7)*
5. **Escrow release authority is platform-held** — operationally trusted. *(hardening)*
6. **Single-instance infra** — rate-limit + metrics + notifications are built (M6), but the lock/limiter/ counters are in-process; **Redis (multi-instance) + real push** are *(M7)*.
7. **Mobile lacks the cash-out order flow** (P2P only). *(parity work)*

Where we are

Committed on branch `sacred`: anchor program, backend (M1–M6), web+mobile + de-crypto, docs. Next and **final** milestone: **M7 — Real rails** (live FX, real payout + KYC/AML vendors, mainnet) — the one that flips every simulated component to production. See 13-future-milestones for the M7 detail and 11-compliance for the risk map.

13 · Future Milestones (M7)

The last unbuilt milestone, documented to the same standard so the full vision is on paper. **M0–M6 are ✅ built** (simulated rails where needed; see 12) — M7 flips every remaining **simulated** edge (11) into a **real, licensed** one.



M7 · Real Rails

What it will be. Flip every simulated component to production: a **live FX source**, **real payout providers** (UPI/Pix/bank), **KYC/AML vendors** live, and **mainnet** USDC (Circle's real mint) instead of a controlled devnet mint. The faucet becomes a **real on-ramp** (card/bank/Apple Pay via a licensed partner). Plus the **infra-scale carry-overs from M6**: **Redis** (multi-instance locks/queues/ quote-cache), a shared rate-limit store, a real metrics scrape target, and **real push/webhook** notification delivery (FCM/APNs) over the in-app inbox.

Why. The endgame: real users moving real money. Everything before M7 exists so that this switch is a *configuration + partnership* change, not a re-architecture.

Legalities. The **regulated core**: licensed on-ramp/payout partners (PSP/MSB/VASP) carrying the fiat legs, live KYC/AML enforcement, FX disclosure, mainnet custody/key hardening (HSM/KMS/multisig for the fee-payer + constrained escrow authority). **Counsel sign-off required before launch.**

Status. ⬜ Planned. Seams are everywhere by design: `USDC_MINT` → Circle's mint; `solana.ts` faucet → on-ramp partner; simulated LP payout → real PSP; devnet → mainnet RPC. **(Live FX already shipped — `fxRate.ts`; what remains is a *regulated* FX provider SLA paired with real payout.)**

The throughline

Every future milestone replaces a **simulated edge** with a **real, licensed one** at a **seam that already exists** in the code. That's the payoff of "build the structure with simulated rails first": the hard architecture is done; what remains is partnerships, vendors, and turning regulated features on in the right order. See 12-status-and-roadmap for the current heat-map.

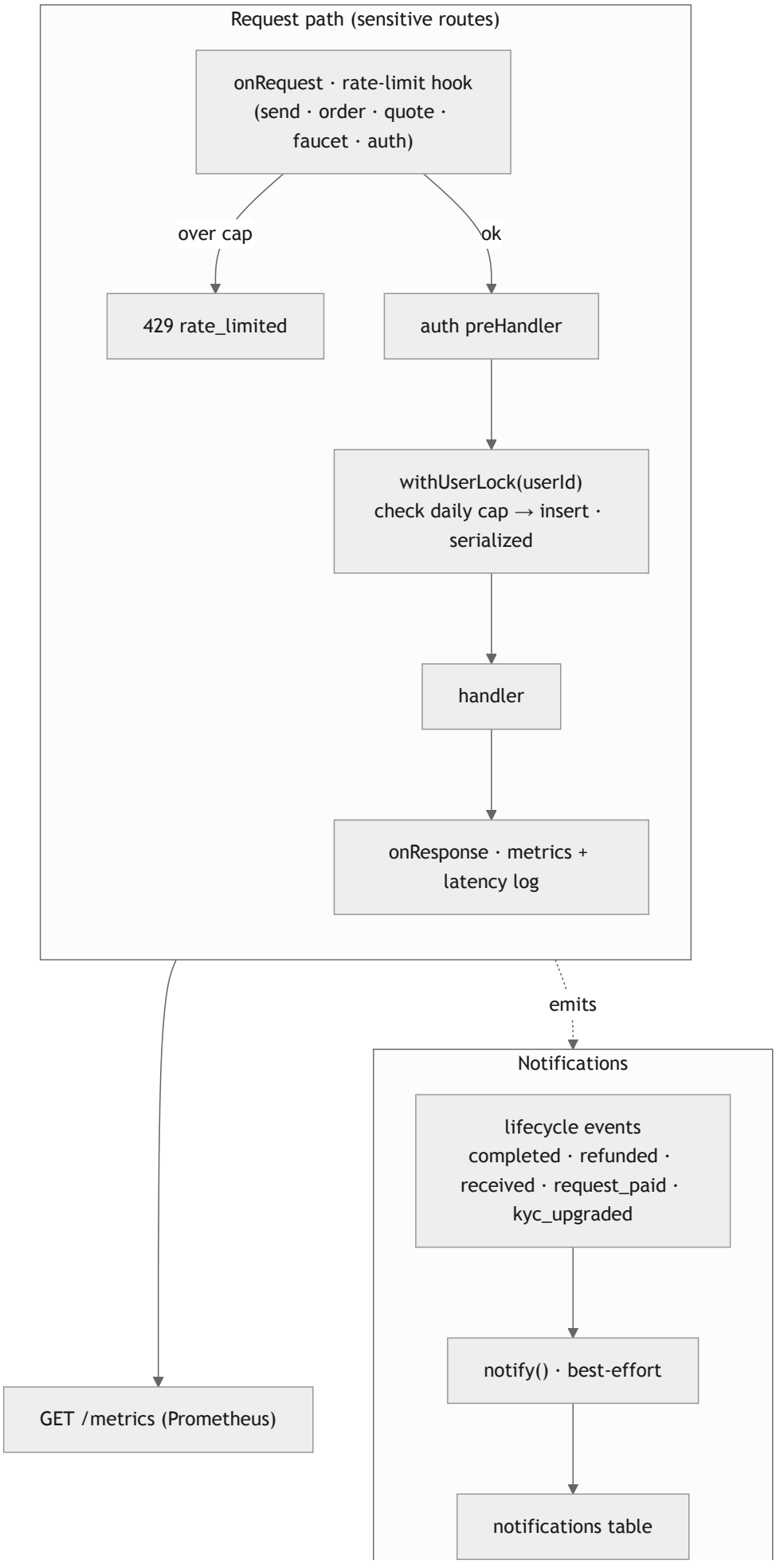
14 · Notifications & Infra Hardening (M6)

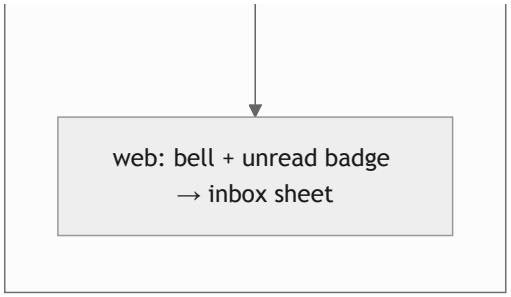
What it is

The "make it production-shaped" layer: tell users what happened to their money, serialize the money-movement checks so they can't be raced, throttle abuse of the expensive endpoints, and expose operational metrics. **M6 builds all four with zero new infrastructure** — no Redis, no queue service, no new npm dependency — using Postgres advisory locks, an in-house rate limiter, and in-memory counters. The pieces that genuinely *need* external infra (real push delivery, multi-instance Redis) are deferred to M7.

1. **In-app notifications** — a `notifications` table + a best-effort `notify()` helper, surfaced as a header **bell + unread badge + inbox sheet** in the web app.
2. **Per-user advisory lock** — `withUserLock(userId, fn)` over `pg_advisory_lock(hashtext(userId))`, closing the M5 daily-cap **TOCTOU**.
3. **Rate limiting** — an in-house fixed-window limiter on money-movement + faucet + auth (→ `429`).
4. **Observability** — in-memory counters at `GET /metrics` (Prometheus text) + a pino latency log.

Flow


Request path (sensitive routes)
onRequest · rate-limit hook (send · order · quote · faucet · auth)
over cap
ok
429 rate_limited
auth preHandler
withUserLock(userId) check daily cap → insert · serialized
handler
onResponse · metrics + latency log
emits
Notifications
lifecycle events completed · refunded · received · request_paid · kyc_upgraded
notify() · best-effort
notifications table
GET /metrics (Prometheus)



The lock closes the TOCTOU. M5 enforced KYC daily caps by reading rolling volume then inserting the transfer/order — two statements. Two concurrent requests could both read "under cap" and both insert, beating the limit. `withUserLock` takes a **session advisory lock keyed on the user** for the whole check→insert, so per user those critical sections run one at a time. Different users never contend (distinct keys). Verified by `check-infra.ts`: two concurrent over-cap $200 sends → exactly **one** passes.

Rate limiting protects the fee-payer. The platform sponsors gas on every transaction and the faucet mints test USDC, so an unthrottled `/devnet/faucet` or `/transfer/prepare` is a drain vector. A fixed window per identity (`x-dev-user-id` / bearer token / IP) caps each — faucet tightest (5/min), sends 20/min — returning `429` with `retry-after`.

Notifications are best-effort. `notify()` swallows its own errors: a notification must never fail a payment. They populate the in-app inbox today; **real push** (FCM/APNs/web-push) and webhooks are M7.

Why this way

- **Advisory locks over Redis.** The correctness need (serialize a per-user critical section) is met by Postgres itself — no new service, no new failure mode. Redis (distributed locks, queues, quote-cache) only becomes necessary at **multi-instance scale**, which is an M7 concern.
- **In-house limiter over a plugin.** A `Map` fixed-window is ~40 lines, no dependency, and sufficient for a single instance (the budget-conscious call). Multi-instance production swaps in `@fastify/rate-limit` + a Redis store — a drop-in at M7.
- **In-memory metrics + pino.** Counters + the structured logger Fastify already ships answer "is it healthy / how much traffic / how many limit blocks" without a TSDB. A real scrape target is M7.
- **Best-effort notifications.** Decoupling the notification write from the money path (try/catch, no await-or-fail) means the feature can never cause a failed payment — the cardinal rule.

Legalities

- **Transaction notifications** are a consumer-protection expectation (and in some regimes a requirement): the user must be told when money leaves or arrives. The inbox is the first cut; auditable delivery (push receipts) is part of the real-rails work.
- **Abuse / fraud controls.** Rate limiting is a baseline AML/operational control — it bounds automated abuse and complements the M5 screening/velocity checks.
- **Observability = operational compliance.** "We can prove what the system did" (metrics + structured logs + the on-chain audit trail) is foundational for incident response and regulator questions.

Status

- ✅ In-app notifications (table + emits at 5 lifecycle points + endpoints + web bell/badge/inbox).
- ✅ Per-user advisory lock closing the M5 daily-cap TOCTOU (verified).
- ✅ In-house rate limiter (`429`) on money-movement + faucet + auth.
- ✅ Metrics at `GET /metrics` (Prometheus) + pino route/status/latency log.
- 🟡 **Single-instance** — the lock uses a **dedicated lock pool** (separate from the query pool, so a held lock can't starve the queries inside its own section); the limiter + metrics are per-process. Multi-instance (Redis locks/queues/cache, shared rate-limit store, a scrape target) and **real push/webhook delivery** are **M7**.

Key files

- `apps/backend/src/services/notify.ts` (emit helpers), `metrics.ts` (counters), `rateLimit.ts` (`rateLimitHook`, `RATE_LIMIT_RULES`, `sweepRateLimitWindows`).
- `apps/backend/src/db/queries.ts` (`withUserLock`, notification CRUD, `getSendNotifyInfo` / `getOrderUserAmount` / `getRequestNotifyInfo`), `db/schema.sql` (`notifications`).
- Emits in `services/{orderMachine,confirmer}.ts`; `routes/index.ts` (rate-limit hook, `/metrics`, `/notifications*`, the two `withUserLock` wraps); `server.ts` (onResponse/onError hooks).
- Web: `components/sheets/notifications.tsx`, `components/SheetHost.tsx`, `app/(tabs)/home/page.tsx` (bell + badge), `store/usePayStore.ts`, `services/api.ts`.
- Harness: `apps/backend/scripts/check-infra.ts`.

Related

09 Confirmation pipeline · 10 Data model · 12 Status & roadmap · 13 Future milestones